

Problem-Sized Lean Worlds

Persistent, checkable research records for AI-assisted mathematics

WILL COOK*

September 2026

ABSTRACT

A proof becomes usable by other mathematicians when it arrives with an exact statement, an explanation of its hard step, its relation to earlier work and a precise account of what remains open. This paper describes a research environment organised around a persistent unit of work, the problem-sized world: the files in a public Git repository that keep one research problem together with its results, failed routes, experiments, literature and exact open obligations. A short paper explains the strongest results to an expert, a long record keeps the complete arguments, Lean checks the formal proofs and Comparator checks selected ones against separately written statements, and a claim registry fixes the wording, evidence class and limits of every public claim. The public record supports continuation by human researchers and compatible agents beyond the originating chat or provider, and accepted work returns to it with credit. A public Lean repository maintains eight Erdős problems this way. Its results include an irrationality criterion for Problem 257, checked in Lean and by Comparator, that admits supports with divergent reciprocal sum, and a formal refutation of the Formal Conjectures statement of Problem 1041, built on a counterexample posted by the forum contributor ani; Google DeepMind's Formal Conjectures now records the answer as False and links the proof. Worked cases from Problems 257 and 249 and from a cross-problem synthesis show what the record keeps for a result, and a transfer between Problems 251 and 269, checked in Lean, shows one problem's theorem proving another's criterion and then improving its constant from $1/31$ to $1/3$. Replaying the recorded failed routes of all eight problems turned a stalled argument into an ordinary proof, so far checked only by a second agent, that the Problem 269 distinct-height sum is irrational for every set of at least two primes, which Erdős asserted without proof in 1973; Lean checks the case of the primes 2, 3 and 5. The record also reads every unfinished argument out of the elaborated Lean environment: a unification search under a time budget finds the theorems that supply, still need or refute each standalone hypothesis, and the kernel checks each composition it proposes and each theorem restated without a hypothesis or with only what its proof uses of it. The resulting argument graph, joined to the papers, the ledger and Comparator, shows what is proved and by which chain, what is refuted, which open statements are one question in other coordinates, what each missing input would settle and what fails without a given lemma. A research record keeps custody of the answers outside research runs return, for the rounds it records, and computes what each still owes. The paper compares the design with Prove2Me and blueprint-based systems, and the argument graph with earlier work on proof generalisation and truth maintenance. A source-bound benchmark tests whether the repository's query routes recover known obligations; it does not measure what a reader learns. We give a controlled reader-study protocol for frontier, status, restatement and transfer decisions. No controlled comparison or prospective discovery study has yet established a benefit from the infrastructure.

**Authorship and AI use.* Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

Contribution and scope

Design. The problem-sized world as the persistent unit of AI-assisted research; four separately recorded decisions about every returned result, on its evidence, its mathematical appraisal, its exposition and the next unit of work; an argument graph of the record's unfinished arguments, read out of the Lean environment and joined to the papers and Comparator; a research record that keeps custody of returned research and what it owes; a path from proof to public claim that separates what Lean settles, what Comparator settles and what a mathematician judges; and a contribution cycle that a person or an agent can run from a public clone. **Instance.** Eight Erdős problems in one Lean repository: sixteen problem papers, 1,903 Lean modules, and 710 results asserted in the papers, short and long versions counted separately, of which 679 carry a Lean declaration and 633 have been compared by Comparator; an export of Lean tree ca2f57c9 gave the argument graph 17,442 theorems, 1,163 of them conditional arguments. **Scope.** For Problem 1041 the Formal Conjectures statement is refuted; independent human review of its correspondence with the 1958 wording has not been recorded. The other seven targets remain open. The system leaves novelty and significance to experts. The current benchmark checks retrieval against known source handles; comparative reader and discovery effects are unmeasured.

1 A theorem and the explanation it needs

Problem 257 asks whether $\sum_{a \in A} (2^a - 1)^{-1}$ is irrational for every infinite set A of positive integers [1]. For an integer $b \geq 2$ write $X_A(b) = \sum_{a \in A} (b^a - 1)^{-1}$, fix a finite nonempty set P of primes, and let $h(a) = \prod_{p \in P} p^{v_p(a)}$ be the P -part of a , where $v_p(a)$ is the exponent of p in a . The Problem 257 short paper [2] proves that

$$\sum_{a \in A} \frac{h(a)}{a(2^{h(a)} - 1)} < \infty$$

makes $X_B(b)$ irrational for every integer $b \geq 2$ and every infinite $B \subseteq A$. Lean proves the criterion, and Comparator has checked that proof against the separately written statement (Section 7).

The criterion reaches past reciprocal summability. Let

$$A_* = \{2^k m : k \geq 1, m \text{ odd}, m \leq 2^{2^k}\}.$$

The k th layer of A_* has reciprocal mass at least $1/4$, so $\sum_{a \in A_*} 1/a$ diverges, while for $P = \{2\}$ an element $2^k m$ contributes $1/[m(2^{2^k} - 1)]$ to the weighted sum and the whole layer at most 2^{1-k} . Every infinite subset of A_* therefore has an irrational value at every integer base. These estimates are ordinary arguments in the short paper; Lean supplies the criterion they feed.

The proof turns rationality into a lattice. For $N \geq 1$,

$$0 < \Delta_A(N) = \sum_{a \in A} \frac{b^{N \bmod a} - 1}{b^a - 1} = (b^N - 1)X_A(b) - J(N), \quad J(N) \in \mathbb{Z},$$

so if $X_A(b) = p/q$ every such displacement is at least $1/q$, and it suffices to find arbitrarily small ones. Sampling N along multiples of a modulus that every element of a finite part of A divides removes that part. The infinite tail is the hard step. Each remaining term is periodic in N ; averaging over the first T multiples of the modulus bounds its complete periods by its weighted summand when the P -part of the exponent divides the modulus, and through a large common divisor otherwise. The leftover incomplete periods of infinitely many exponents need a second average, over dyadic values of T . Its length is set between two scales. Once the finite part of A and its modulus are fixed, the sampling modulus grows polynomially in a parameter H , while

the large-divisor bound has a denominator exponential in H ; an averaging length between the two makes both errors vanish as H grows (`WeightedReturn.lean` carries both averages).

Erdős proved the pairwise-coprime case of the reciprocal-summable criterion and stated, without details, that coprimality can be removed [3, p. 222]. The short paper gives a complete proof of that stated extension at every integer base, which Lean checks and Comparator has compared, and credits it to him. Erdős also remarked that reciprocal summability could be replaced by a weaker, more complicated condition, which he did not state; the short paper does not identify its weighted condition with that remark, and it credits Duverney and Tachiya for a related selection step [4]. The criterion excludes full support and the set of all odd exponents, and irrationality for every infinite support remains open.

Kra writes that solutions arrive “faster than the mathematical community can read them” [5]; Cohn names the technical debt [6]; Tao asks for insight from the process [7]; a declaration signed by twenty-five Fields Medallists calls for writeups, isolated methods and citations [8]; Sanderson writes that “every AI-generated proof is born an unsolved exposition problem” [9]. Each of these concerns what a proof does not carry by itself: failed routes, decisive computations, antecedents and the remaining question.

This paper describes a research environment built to keep that record. Its unit of work is the *problem-sized world*: one research problem together with everything learned about it, kept in a public repository. Section 3 reads the world’s unfinished arguments out of its Lean source, and Section 5 follows returned research into the record.

2 A problem-sized world

A problem-sized world keeps one research question in a versioned public Git repository. Its short paper, 12 to 28 pages long, states the strongest results, mechanisms, antecedents and open boundary. Its long record holds complete arguments, failed routes, bounded experiments, corrections and attribution. Beside them sit Lean source checked by a pinned kernel [10], a claim registry recording each claim’s wording, evidence and limits, and a ledger linking paper results to Lean declarations or recording their absence. A margin mark links a result to its declaration and any Comparator receipt; an unmarked result rests on the paper’s argument, and a dagger marks an input assumed by the formal proof.

Every result that returns to a world raises four decisions (Table 1). A successful check in one row settles nothing in another: a theorem may belong only in the long record, a conjecture may guide a computation, and clear prose may describe the wrong formal statement.

Problem 249 asks whether $S = \sum_{n \geq 1} \varphi(n)/2^n$ is irrational, where φ is Euler’s totient function [11]. Its short paper [12] starts from an exact theorem about the subsequences obtained by splitting the coefficients of S into residue classes modulo powers of a base. With $\mathbb{N} = \{0, 1, 2, \dots\}$ and $\varphi(0) = 0$, the seven sections $\varphi(2^j n + r)$ with $0 \leq j \leq 2$ and $0 \leq r < 2^j$ reduce to the five sequences

$$\varphi(n), \quad \varphi(2n), \quad \varphi(2n+1), \quad \varphi(4n+1), \quad \varphi(4n+3),$$

since $\varphi(4n) = 2\varphi(2n)$ and $\varphi(4n+2) = \varphi(2n+1)$. These five are linearly independent, and the pattern holds in every base. For every base $k \geq 2$ and level $e \geq 1$, the sections $\varphi(k^j n + r)$ with $0 \leq j \leq e$ and $0 \leq r < k^j$ span a space of dimension $k^e + 1$ over $\mathbb{Q}$, with basis $\varphi(n)$, $\varphi(kn)$ and the sections with $1 \leq j \leq e$ and $k \nmid r$; every other section is an explicit integer multiple of a basis element, and these reductions generate all integral relations. Coons proved that φ is not k -regular for any $k \geq 2$ [13]; the basis theorem recovers this and gives the dimension at every level.

Lean proves the theorem for every base, and Comparator has compared it. The reductions are elementary; the independence carries the weight, through an evaluation matrix made diagonal and nonzero modulo an auxiliary prime, with evaluation points chosen by the Chinese

Decision	What the world records	Weighted criterion	Dyadic tail differences
Evidence	The exact statement, its hypotheses and named inputs, the Lean declaration and its checking receipts	Lean proves the criterion; Comparator has compared the declarations that state it	Lean proves the equivalence with $1/31$ and then with $1/3$; the application trace records the rejected and the accepted step
Appraisal	The mechanism, the closest antecedent, the strength of the result and the obstruction that survives	Two averages; Erdős's reciprocal-summable case, credited; admits A_* ; excludes full support	A change of state variable; $1/31$ was inherited from radix 30, and $1/3$ cannot be raised for radix-two orbits in general
Exposition	Whether the result belongs in the short paper or the long record, and how its hard step is explained	The theorem opens the short paper; the example and the full averaging proof follow it	Section 4 shows the whole transfer; the experiment keeps the trace
Allocation	The next question, the test that would decide it and the condition for stopping	Irrationality for every infinite support stays the registered endpoint; a changed-base reader exercise and independent replay route now test outside use of the weighted condition	Is $1/3$ best for irrational tails? A probe pointed to 0.4125, the literature fixed the answer, and the question closed. The restricted-shift theorem and reader exercise now test whether an outsider can use that answer; outside use has not yet been observed

Table 1: Four decisions about a returned result, for the weighted criterion of Section 1 and the cross-problem transfer of Section 4. An enduring endpoint and a justified next step are different allocations.

remainder theorem and Dirichlet's theorem so that each row asks one primitive affine form to take a prime value. Martin's Theorem 1 already implies the affine independence [14], and the separation method goes back to Yazdani, who credits it to Shallit [15]; the short paper credits both. The record keeps the basis theorem, a statement about the coefficients, apart from the irrationality of S , whose open form it states exactly.

Write $R_N = \sum_{j \geq 1} \varphi(N+j)2^{-j}$ for the tail of S after N terms, put $H_t = \text{lcm}(1, \dots, t)$, and write $\text{Cert}(t)$ for a finite exact computation, on totient values just after H_t and $2H_t$, showing that $R_{2H_t} - R_{H_t}$ is not an integer. Lean checks, and Comparator has compared,

$$\forall t \leq 82, \quad \text{Cert}(t),$$

and Lean also proves, again with a Comparator comparison, that irrationality of S is equivalent to the unbounded supply

$$\forall T, \exists t > T, \quad \text{Cert}(t).$$

The finite theorem makes no $t = 83$ or cofinal claim. What remains open is the unbounded supply itself, which is the irrationality of S in equivalent form, and the record states it in exactly that form.

3 The argument graph

The part of a record that crosses least well from one reader to the next is the unfinished argument. A theorem $T : \forall \rho, H_1 \rightarrow \dots \rightarrow H_k \rightarrow G$ whose hypotheses nothing proves establishes G once the H_i are supplied, and a library written during research holds many of them, each waiting for an input that another module may already prove, refute or restate. A hypothesis that mentions none of the earlier binders is a *closed statement*, a proposition in its own right. A data binder the conclusion does not mention is a hypothesis too: when its type is closed and not known to be inhabited, the theorem needs the closed statement `Nonempty  $\tau$` . The record reads every theorem out of the elaborated Lean environment, the two library roots together with every module the coverage build compiles, and connects these statements into an *argument graph* ([docs/ARGUMENT_GRAPH.md](#)).

Export. A Lean program ([export_argument_continuations.lean](#)) runs six passes in continuous integration. It records theorem hypotheses and what each proof uses of a closed hypothesis $h : H$. If every use of h is an application mentioning no other variable of the theorem or its proof, the propositions these applications prove replace H ; the kernel checks that theorem and each implication from H . These are its *used consequences*. For each closed statement, a budgeted search unifies corpus conclusions with it under definitional equality. It introduces the statement's binders, so $\forall n \geq 7, P(n)$ can use a theorem deriving $P(n)$ from $n \geq 7$; it follows proved equivalences, conjunction components and negations, discharges producer premises and searches the remaining obligations. A producer needing an undetermined witness of a type not known inhabited is recorded but cannot supply the statement. The export also searches for refutations, composes conditional theorems whose closed hypotheses are all supplied, drops unused proposition binders, and tries bounded closing tactics and library search, including on negations. A composition, a stronger statement, a weakened theorem or a tactic proof counts only if the kernel accepts it. An exhausted budget is never read as the absence of a producer.

The graph. A *reduction* of S is a theorem that supplies S once its remaining hypotheses are supplied. Table 2 defines the rest. Two further checks are derivations over recorded edges: a statement is refuted by conflict when supplying it would supply both a statement and the other hypotheses of a recorded refutation of that statement, and a bundle is dropped when its members are jointly impossible and marked as the target restated when the target implies each member. The graph joins theorems to citing paper results, their $\text{\TeX}$ lines and Comparator status; it lists paper results assuming each open statement. The builder checks that authored conditional implications and frontier antecedents still depend on open statements.

What it shows. The numbers below come from two exports of the record's Lean sources at an earlier revision (Lean tree `ca2f57c9`), the second with the used-consequence pass, combined because every row of both observes the same elaborated environment. Each search stopped at its time budget before reaching most of the statements it had discovered. The graph has 119,284 statements from 17,442 theorems in 1,457 Lean modules, of which 1,163 are conditional arguments. 5,672 statements are supplied. 196 are refuted by a corpus theorem or a tactic proof of their negation, 1 by a reduction to a refuted statement and others by conflict, and the refutations make 0 theorems vacuous. 113,362 statements are open, a count that includes every statement neither search reached. The kernel accepted 18 compositions, 154 statements without a hypothesis their proofs never use, 0 of them dropping a hypothesis that is open, and 89 tactic proofs of statements for which the search found no producer. The open statements fall into 2,136 disguise classes, the largest with 36 members. 316 theorems use a closed hypothesis only through consequences of it, 302 of them an open hypothesis, and 8 statements are supplied only through such a replacement.

Notion	Definition
Supplied	Some reduction has every remaining hypothesis supplied (least fixpoint, so a cycle of reductions supplies nothing by itself); the first such reduction is kept as a witness chain of kernel-checked theorems
Refuted	A theorem proving the negation has every remaining hypothesis supplied, or a tactic proves the negation; refutation passes back along a reduction whose other hypotheses are supplied
Vacuous theorem	A conditional theorem with a refuted closed hypothesis
Idle hypothesis	A hypothesis the proof never uses; the kernel accepts the statement without it
Used consequence	A proposition the proof derives from a hypothesis and uses in its place; the kernel accepts the theorem with the hypothesis replaced by its used consequences, and a refuted consequence refutes the hypothesis
Disguise class	Open statements that imply one another: one open question in several coordinates
Bundle	A set of open statements whose supply supplies a given statement, minimal among those a bounded search finds
Leverage	The open statements that become supplied when a given statement is supplied
Criticality	The supplied statements that lose every chain when a given statement or theorem is withdrawn

Table 2: The notions the argument graph computes from the relations the export records.

What a proof uses of its input. The Problem 249 long record proves an excluded-cofactor estimate, a bound $(D + o(1))X$ on a count at scale X in which D is the density of the integers m with $\varphi(m) < \eta m$; the argument needs the count below $X/100$. The Lean proof ([Excluded-CofactorEstimate.lean](#)) takes the prime number theorem as a named input, and the paper marks the result with a dagger. In a kernel probe the used-consequence pass shows that the proof uses it only through a chain of lemmas ending at the dyadic prime count $\#\{p \text{ prime} : a < p \leq 2a + 1\} \cdot \log a \leq (1 + \varepsilon)a$ for all large a , and the kernel accepted every weakened theorem on the chain. The argument is linear in the constant K in place of 1 (that step was by hand) and bounds the count by $(KD + o(1))X$. Chebyshev's bound, that the primes in $(a + 1, 2a + 1]$ divide $\binom{2a+1}{a} \leq 4^a$, gives $K = \log 4$; at $\eta = 1/1000$ an elementary estimate gives $D \leq 3/1000$, and $3 \log 4/1000 < 1/100$, so the count stays below $X/100$ with no input at all, as it does for every $\eta > 0$ with $3\eta \log 4 < 1/100$ ([excluded_budget_of_chebyshev](#)). With the unconditional `prop_dickman` the count discharges the depth, bad-base and non-supplier conditions of Problem 249's first-harmonic route, leaving its fibre-mean and centred estimates ([Results/Erdos249.lean](#)); the prime number theorem gives the fibre means, and on the good bases the centred estimate is the first-harmonic gap up to $X/100$ ([Results/Erdos249Route.lean](#)). The irrationality needs less: a set of bases whose average first cosine is at most $\frac{9}{10}$ contains a certified kill, and more than $67X/100$ of the bases are good, so the good-base gap at $603X/1000$ alone gives it ([Results/Erdos249Endpoint.lean](#)). A search command, `residualise`, finds the route through the decorrelation with the composition withheld, fixing the depth and η by unification,

and adds once the kernel accepts it the theorem that the clauses no supplier gives imply the decorrelation: with the fibre-mean theorem among the suppliers, the prime number theorem and the centred estimate ([Results/Erdos249Residual.lean](#)).

The findings as library theorems. A Lean module ([Derive.lean](#)) turns these findings into theorems that a paper or another proof can cite by name. Each command reads a proof term and adds, once the kernel accepts it, the theorem without the hypotheses its proof never uses (`derive_idle`), with a hypothesis replaced by what its use sites prove (`derive_weakening`) or by where the chain of such replacements stops, optionally at a named lemma and discharging what named corpus theorems prove (`derive_frontier`), or each conjunct of the conclusion with only the hypotheses its own proof uses (`derive_conjuncts`). A generator ([build_argument_frontier.py](#)) writes, from Lean tree 6410fad4, the commands for 222 cited theorems, 45 of them with a hypothesis the proof never uses, in 8 modules, one per problem, and the build fails when a derivation the export reported cannot be rebuilt. In kernel probes for Problem 249, the first two conjuncts of the excluded-cofactor estimate hold with neither the prime number theorem nor the density hypothesis, and the Lean proof that $\sum_{m \geq 1} \sigma(m)/2^m$ is transcendental uses Nesterenko's theorem only at $q = 1/2$. For Problem 1041, the separation corollary holds without the hypothesis that the roots lie in the open unit disc, which neither its proof nor the separation theorem it assumes uses. A derived theorem restates what its source proof already proves. 73 statements the graph leaves open occur 129 times among the hypotheses of the cited theorems; a proof of one removes it from every paper result that assumes it. 71 of the cited theorems have a weakening, with 9 frontiers cut at a named lemma, and for 55 of the open hypotheses the graph records what the proofs use of them, level by level. In 2 frontiers the named corpus theorems prove all that the proof uses of a hypothesis, so the derived theorem drops it: Problem 251's equivalence between the irrationality of the prime and gap series holds without the summability hypothesis of its Lean theorem ([Erdos251.lean](#)), and Problem 257's square-root route without its sign hypothesis ([Erdos257.lean](#)).

A second module ([Factor.lean](#)) moves each hypothesis of a theorem to the claims of its conclusion whose proofs use it and keeps the proof's witnesses (`derive_factor`): from a proof of $H_1 \rightarrow H_2 \rightarrow \exists w, A(w) \wedge B(w) \wedge C(w)$ whose witness and whose proof of A use neither hypothesis, it states $\exists w, A(w) \wedge (H_1 \rightarrow B(w)) \wedge (H_2 \rightarrow C(w))$. When the proof deconstructs a corpus lemma applied to a hypothesis being moved, it factors that lemma first, so it follows a construction through the lemmas it is built from. The kernel checks every theorem it adds. Problem 251's simultaneous countermodel assumes Schlage-Puchta's Lemma 4 and the prime number theorem and fixes $0 < \varepsilon \leq 1$; its factoring gives the construction for every $\varepsilon > 0$ with each input assumed only by its own clause ([Results/Erdos251.lean](#)), and the exponents $\varepsilon > 1$ add nothing, since the sequence for $\varepsilon = 1$ serves them. Following the lemmas can drop a hypothesis the proof as written uses: Problem 243's transport-square theorem passes $v_n > 0$ only to claims it does not use, and the other hypotheses force $v_n > 0$ anyway ([Results/Erdos243.lean](#)), so the factored theorem is the same result, stated shorter. Problem 1041's inner-chord factoring drops a switch condition; off it the maximality clause holds vacuously and the midpoint equality is asserted, as at $n = 3, r = 1/2$ ([Results/Erdos1041.lean](#)). Model checking draws these distinctions: a subformula is vacuous if replacing it by false keeps a passing verdict [54]. 122 cited theorems have a factoring, and deriving them factored 69 lemmas beneath them. A probe credits a command only with the theorem it names, never with lemma factorings derived on the way. The generator carries every published command into each new plan and stops when a probe shows a published name no longer added or stating something else, so a later export with a smaller budget or another use-site rule cannot remove a theorem a paper cites. The check has stopped one regeneration, in which ten published frontiers would have followed parameterised weakenings to stronger statements; a frontier follows only the kind of weakening it was published with.

Research with the graph. A bounded problem packet ([query_continuations.py](#)) lists open targets, reductions, paper consumers, disguise classes, multi-member bundles, refutations, unused hypotheses, used consequences and leverage. Before attacking an intermediate statement, a reader can ask whether the corpus already supplies or refutes it, or whether it restates the target. Proof terms also show which theorems consume a hypothesis, what they use of it and which chains fail if it is withdrawn.

A session rarely has a corpus build, which needs several gigabytes and, without a cache, hours. It writes the statement it wants checked, with its proof, as a scratch Lean file on a probe branch; continuous integration builds the corpus from the main branch's cache, runs the file and returns a verdict, which accepts the file when Lean reports no error, no declaration uses `sorry` and no axiom beyond the three standard ones is declared or printed ([kernel-probe.yml](#)). The verdict is a compilation receipt: it does not check that the file states the intended target, and the workflow keeps it for fourteen days, so a verdict cited later must be committed. A route that a countermodel is suspected to block is tested the same way, by stating the route as a proposition and proving its negation from the countermodel.

What it does not establish. The kernel-checked compositions, stronger statements, weakened theorems and tactic proofs are proofs; a supplied or refuted statement rests on kernel theorems joined by the unifier; the rest of the graph is a map, and a lower bound. The search tries only corpus theorems as producers and prefilters them by the constants in their conclusions, so a statement shown open may follow from an argument the search did not try, or may not have been searched at all. A disguise class records proved equivalences only, and leverage counts consequences inside this record. Importance, difficulty and interest remain judgements for a mathematician. Before any number above was reported, an adversarial review of the graph code found confirmed defects, among them one that counted a theorem needing a witness of an empty type as unconditional and so reported `False` as supplied. On the stream exported before the review, the same defect also supplied `1/2` and `1/21` as members of the Problem 257 achievement set and the irrationality of the Problem 249 series, through producers that need a certificate no one has constructed. Each defect is now a regression case in the exporter's test corpus, and the builder stops with an alarm, before any number reaches a paper, whenever `False` or a registered open target comes out settled.

4 From corpus to insight

The short papers are distilled from a much larger research record. Each problem keeps a research corpus of exact results, negative results, computations, literature notes and proposed next steps, including intermediate and superseded entries, and the mathematical loop runs over it in two coupled roles.

Discovery and stewardship. A discovery pass stays close to the frontier: it runs discriminating computations, attempts proofs and returns results. A stewardship pass reads each new result against the whole corpus and makes the four decisions of Table 1: what the result establishes, whether it is routine or strong and whom it credits, where it belongs in a paper, and which obligation deserves the next unit of work. One agent may play both roles in sequence. Neither inherits the other's authority.

Models do the reading, comparison and drafting under written procedures, published with the repository as agent skills. Deterministic code assembles each long record from its authored parts, records the Lean evidence of every asserted result or its absence, and checks that each result a paper states is linked and that each registered limitation stays visible. No deterministic check decides what is important; that judgement, whoever drafts it, is written into the paper, where a reader can inspect and dispute it.

Testing a proposed measure of value. Patel et al. [30] propose conditional proof difficulty as a way to choose useful mathematics: a lemma is valuable when making it available shortens later proofs. Leverage in the argument graph is its logical counterpart, the open statements a lemma would settle, and like their score it is an input to an allocation. Our [accompanying experiment](#) tests two proxies for their score on this record: over 9,289 cited declarations they have Spearman correlation 0.799, yet all 64 fixed-seed shuffles of the user counts give a larger correlation, so the shared proof-cost factor, and not observed reuse, carries it. The [stewardship procedure](#) requires a named consumer or falsifiable test and records the allocation separately from any score.

Negative results are part of what the loop produces. Tao observes that withheld negative results and processes hide where a new frontier lies [7]; the record keeps them. A counterexample refutes a conjecture on its exact domain, and a Lean no-go theorem rules out a class of strategies under explicit hypotheses, so later work need not repeat either. A failed computation records its finite domain and the inference it defeats. For Problem 257, a proposed estimate said that the greedy test ruling out a rational number with denominator Q as the sum of an infinite subseries of $\sum_n 1/(2^n - 1)$ stops within about $2 \log_2 Q - 3.3$ steps; an [exact rational probe](#) first rules out $189/388$ at step 17. The record keeps both facts: the estimate is false, and the problem itself is untouched. Inside the weighted proof the same discipline explains the second average: the short paper shows why the majorant of the reciprocal-summable proof fails beyond that class.

Reading the problems together raises questions that none of them asks alone. The smallest complete case is a transfer between two problems. For dyadic tails $T_{N+1} = 2T_N - g_{N+1}$ with integer digits g_N , the Problem 251 record proves that T_0 is irrational exactly when no difference $T_{N+h} - T_N$ with $h > 0$ is an integer. The Problem 269 record proves an escape theorem: an orbit $x_{n+1} = p_n x_n - c_n$ with $p_n, c_n \in \mathbb{Z}$ and $2 \leq p_n \leq 30$ reaches an integer or returns infinitely often to distance at least $1/31$ from the integers. Applied to T , the escape theorem says nothing about differences. The difference $U_N = T_{N+h} - T_N$ satisfies $U_{N+1} = 2U_N - (g_{N+h+1} - g_{N+1})$, so it applies to U , and the Problem 251 criterion excludes the integral alternative. Hence T_0 is irrational exactly when every such difference is at least $1/31$ from the integers for infinitely many N . The [experiment](#) imports only the two sources and records the Lean outcomes: the application to T is rejected, the application to U leaves one goal, and the Problem 251 theorem closes it. The change of variable is the mathematical bridge, and Lean establishes what follows. The constant $1/31$ came from the radix bound 30. With the bound B as a parameter the same proof gives $1/(B+1)$, which the orbit alternating between $1/(B+1)$ and $B/(B+1)$ shows cannot be improved in general, and hence $1/3$ for the differences. Lean checks all three, and the $1/31$ run stays in the record as the original outcome. A finite probe then asked whether $1/3$ is best for irrational tails, and a published theorem answered it. Dubickas proved $\limsup_n \|2^n \xi\| \geq \tau$ for every irrational ξ , with equality at the Thue–Morse number $\tau = 0.412454\dots$, as stated by Akiyama and Kaneko [39], so every fixed $0 < c < \tau$ works in the equivalence. The endpoint does not: the doubling orbit of τ stays strictly closer than τ to the integers at every positive index, so the admissible constants form the open interval $(0, \tau)$. The record keeps the Lean-checked $1/3$, the cited theorem and the ordinary endpoint argument, in the [recorded closure](#), at their own evidence classes. This sharpens a quantitative irrationality criterion; establishing the criterion for the actual prime gaps remains open.

The transfer also makes conditional proof cost concrete: with the target fixed, the source work counted when the recorded proof is expanded through the shift criterion or the escape lemma can be compared with the work counted when these are available premises. Such an expansion is an inspectable accounting model; it gives no evidence for the optimal-intermediate assumption behind Patel et al.’s Bellman-type equality [30, Definition 1].

The sharp-constant question then leads to a question about the shifts being tested. A rational dyadic orbit is eventually periodic modulo one, so a family containing a multiple of every positive integer detects its eventual integral differences. The [synthesis theorem](#) proves the converse

with explicit rational counterexamples: factorial shifts suffice, while power-of-two shifts do not. The [reader task](#) asks which shifts still work and why a limsup bound does not include its end-point. The full restricted-shift classification is an ordinary argument using the cited Dubickas bound; the Lean evidence currently covers the 1/3 all-shifts criterion.

Seven of the eight problems ask whether a series is irrational, and the synthesis paper [16] studies what their methods share and where each stops. The sparse corrections used for Problem 251 led to a capacity criterion for factorial digits, and then to a question: can one sequence of factorial digits prescribe several derivatives at once? For $c > 0$, let $\mathcal{H}_c$ be the class of entire functions $f(z) = \sum_{n \geq 1} e_n z^n / n!$ with $e_n \in \mathbb{Z}_{\geq 0}$, $e_n \leq n^c$ eventually, and every fixed positive integer dividing e_n for all large n , and for an integer $d \geq 1$ put $J_d(f) = (f(1), f'(1), \dots, f^{(d-1)}(1))$. The synthesis paper proves

$$\dim_{\mathbb{H}}(J_d(f) : f \in \mathcal{H}_c) = \min(c, d),$$

that this set contains a nonempty open subset of $\mathbb{R}^d$ exactly when $c > d$, and that a nonpolynomial member of $\mathcal{H}_c$ with $f(1), \dots, f^{(d-1)}(1)$ all rational exists exactly when $c > d$. So a nonpolynomial $f \in \mathcal{H}_2$ with $f(1)$ rational has $f'(1)$ irrational, while coefficients of size $n^{2+\varepsilon}$ allow both to be rational. The construction rests on a carry identity,

$$(z-1) \sum_{n \geq 0} \frac{b_n z^n}{n!} = -b_0 + \sum_{n \geq 1} \frac{(nb_{n-1} - b_n) z^n}{n!} :$$

adding such a term to f leaves $f(1)$ unchanged and moves $f'(1)$ by $\sum_n b_n / n!$, and a factor $(z-1)^k$ leaves $f(1), \dots, f^{(k-1)}(1)$ unchanged at a cost of k powers of n in the coefficient allowance. Division by $(z-1)^d$ gives the matching obstruction: vanishing derivatives become bounded integer coefficients, which eventual divisibility forces to vanish. Both theorems are proved in the synthesis paper by ordinary argument and carry no Lean mark; Lean checks the finite carry identity and its preservation of divisibility.

The same corpus reading can suggest a representation absent from the recorded dependency graph. For a divisibility chain of exponents, the synthesis paper previously used a Mahler equation only when successive ratios eventually equal two. Grouping a repeated block of integer ratios instead proves that *every eventually periodic ratio chain has a transcendental Lambert subsum at every algebraic real base greater than one*. For alternating ratios 2, 3, the support is $\{6^k, 2 \cdot 6^k : k \geq 0\}$ and its generating function satisfies

$$G(z) - G(z^6) = \frac{z}{1-z} + \frac{z^2}{1-z^2}.$$

The old doubling equation fails at coefficient four. The block decomposition proves the new equation at every degree; infinitely many boundary singularities and Nishioka's value theorem supply transcendence. This covers base 4/3, where the earlier denominator estimate fails. The full ordinary proof and attribution are in the updated synthesis theorem [16]; the [coefficient probe](#) also rejects the same block equation on a specified nonperiodic ratio word. Finite agreement tests the proposed representation, and arbitrary nonperiodic chains remain beyond this argument. This is an application of classical Mahler theory, with no Lean or novelty claim. It illustrates a candidate-generating move, looking for a functional equation before improving the estimate. The result is not attributed to a numeric ranking, and no measured discovery advantage is claimed.

5 The research loop as it runs

Results enter through a loop between the repository's maintainer and outside research runs. The maintainer chooses a question, gives a run a fixed packet of source excerpts and keeps its return

before review. Claims then face the relevant check: a kernel probe, corpus build, Comparator, rerun computation or reading of an ordinary argument. Surviving work enters Lean, papers and the claim registry; the Problem 249 endpoint theorems in Section 3 came from one such round. Question selection, handoff and deciding what an answer means are manual steps.

The research record. Since 28 September the loop keeps its own [record](#). An append-only hash-linked journal records each packet, return, review, component disposition and declared consumer. Paper links, claim entries and Comparator listings are computed from the checkout with their evidence classes; no field of the record confers kernel authority. Round six’s three infrastructure returns supplied ideas for consumer dispositions, computed milestones, relation contracts, the contrast ledger and packet compiler. The contracts, in Lean ([Contracts.lean](#)), compare two statements inside one context. A *slot replacement* R for a hypothesis H comes with a proof of H from R , so it never counts as weakening H ; a proof of the converse makes the two equivalent, and strictness needs a separating instance. An *endpoint route* proves the endpoint itself from R , and only such a route can reach an endpoint from less than an earlier argument demanded, as the Problem 249 good-base gap of Section 3 does. Equivalent formulations count as one question for credit and stay separate views for search. The contrast ledger records 14 misread distinctions with source spans and review checks. The packet compiler adds triggered contrasts to an author’s obligations within a byte budget. It reports a minimal completed search, an incomplete search or an infeasible budget. A trigger asks for both sides to be checked; it does not reject a result. Restatement requires matching hypotheses and implication directions, with evidence for claimed strictness.

What the record measures. These numbers are what the record’s commands (`research_record.py status`, `relation_registry.py check`, `contrast_ledger.py triggered`) report on this corpus, each at its own evidence class. At journal head `e206968c9dfe` the record holds 7 rounds (2 sealed), entered on 28 September from custody kept before the journal and the packet compiler existed. Of their 25 returns, 18 are admitted for integration with their consumer dispositions incomplete, and 7 are kept with no recorded review. When the record was first filled, its computed milestones showed the two Problem 249 endpoint theorems in a compiled module of the pinned Lean source, with no paper link, claim entry or Comparator listing, and the $\{2, 3, 5\}$ case of the Problem 269 theorem linked from both papers with no claim entry or Comparator listing. All 8 relation rows are attested: the declaration atlas of the pinned source lists each certificate with its recorded signature, in a module that the default build roots or the coverage build reach. Matched against the three published round-six answers, from which most contrasts were drawn, the ledger’s triggers flag five, six and eight contrasts; that exercises the matching and predicts nothing about later answers. These counts describe the process and measure no effect of the loop on the mathematics.

Replaying the record. Zheng et al. [25] store a discovery run as a tree of evaluated attempts, so that another exploration policy can be replayed over it without rerunning any attempt. On 26 September four readers replayed the recorded routes of the eight worlds ([replay](#)), classing 70 decisive rows as proofs, obstructions, counterexamples, finite computations, stalled attempts, superseded formulations or routes never tried; a checker rereads every cited record at the pinned commit. Reopened routes gave the irrationality of the Problem 269 distinct-height sum $\mathcal{D}_P$ for every set P of at least two primes and of each single-prime sub-sum E_p [28]; for two primes this already follows from Hecke–Mahler transcendence [26, 27]. They also gave transcendence of the Lambert subsum over every infinite divisibility chain at every rational base [16]. Lean checks $\mathcal{D}_{\{2,3,5\}}$; the other proofs are ordinary arguments awaiting specialist review, and the replay compares no exploration policies.

6 Testing decisions made from the record

The record has two different possible benefits. It may help a reader identify the current frontier and avoid counting a restatement as progress; it may also help a researcher find an application that would otherwise be missed. The first is a decision about existing evidence. The second requires a new, independently checked consumer. The transfer in Section 4 shows what mathematical reuse looks like, but its presence in the corpus does not measure either effect of the record.

The current `benchmark_semantic_reasoning.py` exercises fixed queries against named source references: it checks that a returned packet contains specified results, statuses and dependency entries. This is a regression test of navigation, not an experiment with readers. An exact statement appearing in two places may be equivalent, strictly stronger in one context, or simply related by an unproved converse; matching words or graph proximity cannot decide which. The relation registry and contrast ledger expose the evidence for review. The journal records the reviewed disposition of each returned component. A measurement of rediscovery must first join companion prose and code belonging to one submitted attempt; files or equivalent formulations are not separate discoveries.

A controlled model-reader comparison would freeze a source revision, task families, the correct frontier, answer keys and grading rules before runs begin. The planned tasks ask for a maximal recorded result under stated assumptions, an exact status, a scoped relation or a possible transfer. Papers alone, the active repository, a compiled context packet and the same excerpts without the packet's structure are four proposed reading conditions. The papers and repository conditions must use current material, and every condition must receive the same correct frontier when the question is whether an explanation or obligation list helps. Hold out tasks by proof and definition lineage, including aliases and generated graph or paper views that could reveal the answer. A task family, not each rephrased prompt or repeated seed, is the independent comparison unit.

Twelve candidate questions span four dependent clusters and thirty-four hash-bound excerpts at one frozen revision. The blind review export includes source bytes; a trusted channel pins its manifest. These are development materials, not unseen gold: hashes and author labels establish neither interpretation nor independent correctness.

The `reader_study.py` builds allowlisted, digest-bound jobs and blind grading packets. Its controller permits an authorized provider bridge only bounded listing, reading and literal search; it records traces, output and usage. The bridge and host must enforce model identity, resource limits and isolation. A four-task, one-family pilot is specified but unrun and ungraded. Excluding answer-bearing aliases and adjudicating keys need independent review. A completed comparison would report every issued task, including timeouts and invalid answers, paired exact-decision accuracy, severe errors, source-use and total preparation and review cost. Independent grading must preserve justified uncertainty when a converse, witness or source claim is missing. No such completed comparison is reported here.

Discovery needs a separate prospective test. Before a target consumer is revealed, freeze the mechanism records and source boundary, then ask whether a candidate map discharges its typed hypotheses, yields a kernel-checked or otherwise fully argued new application, survives a search for existing work, and helps an independent consumer. Record failed probes, human integration time and adverse cases as well as successes. A source mutation, such as withdrawing a guard or correcting a stale statement, offers another test: does the changed claim reach the packet, the reader's decision and each downstream paper or claim without invalidating unrelated work? These are proposed measurements, not results inferred from the number of files, the eight problem worlds or the existing research rounds.

7 From a proof to a public claim

A proof becomes a public claim by crossing three questions that the word “verified” tends to merge: whether the proof establishes its formal statement, which the pinned Lean kernel answers in continuous integration; whether the proved statement is exactly the separately written one, using only permitted axioms, which Comparator answers for selected statements; and whether the formal statement says what the problem says and whether the result matters, which a mathematician answers.

Lean verifies that a proof establishes the formal statement written in the source; it does not verify whether that statement captures the intended mathematics or whether the paper describes it well. The gap is measurable. Feng et al. had mathematicians grade 200 candidate solutions to Erdős problems produced by Aletheia, an agent built on Gemini Deep Think: 63 were technically correct, of which 13 were meaningfully correct; the other 50 answered a reading of the problem that missed Erdős’s intent [17].

Comparator, a checker the Lean FRO built for judging machine-written proofs [18], answers the second question. A trusted challenge module restates each statement without its proof; the proof must be a term of exactly that type, using only the permitted axioms, and the Lean kernel must accept it. The corpus replay, run in the companion repository *plectis-erdos-lean*, also passes every compared proof through *nanoda*, an independent kernel [19]. In this repository, continuous integration also requires Comparator to reject a deliberately altered statement. This control exercises that mismatch path; the surrounding invocation and verdict contract have separate obligations. Comparator cannot decide whether a statement is the right translation of the problem. It guarantees that the separately written statement, which a reader can inspect, is the statement that was proved, and the record reports exactly that scope as Comparator-checked.

The paper-to-Lean ledger makes both formal questions countable for the papers’ own statements. It holds one row for each result asserted in the sixteen problem papers, 710 in all, counting a result stated in both a short paper and a long record twice, with its statement digest and kind of correspondence. For 511 rows the named Lean declarations state the result exactly; for 143 they state it or a result that implies it by an immediate specialisation; 25 are proved from a named input, marked with a dagger; and 31 have no Lean statement. Comparator has compared 633 of the 654 exact and specialising rows, and the other 21 are queued for its next replay; every receipt in the pinned corpus replay was accepted by both the Lean kernel and *nanoda*. A digest detects a changed statement, and the classification of a correspondence stays the maintainer’s judgement.

The third question stays with people. The claim registry records the wording of each public claim with its evidence class, the bounded domain of each finite instance and the open statements it approaches, and the papers, README and generated views may say only what the registry allows. In this repository the maintainer is responsible for that wording, and no independent mathematical review is yet recorded. A release checker keeps those surfaces consistent with the registry and the Lean source, and requires a registered limitation to stay visible wherever its claim appears.

That requirement comes from a test in which a limitation escaped. One deliberately false README edit changed a clause saying that the finite cases of Problem 249 did not supply the open requirement into one saying that they completed it. Lean was untouched, and the checker passed because that relationship had not been registered. After registration, the post-repair witness accepts the current README and rejects a test copy containing the false clause. A small historical study tested the checker with deliberately false edits: nine of the ten edits were rejected. One escaped, for the reason just given. The edits were authored by the checker’s author. The original run logs were not retained. The other nine edits were not rerun against the extended

checklist. The study locates a coverage boundary. Its nine rejections do not estimate how reliable the checker is.

The workflow does not technically force a second independent mathematician: the maintainer can edit a Lean statement, its record and its prose together so that every comparison agrees with the same mistake. Independent review is recorded as an external event when it happens. A changed result reopens what depends on it. Propagation starts from the changed object and lists every reverse import, claim record, paper, experiment and open obligation the change may affect; each must be updated, verified unchanged, deferred with a reason or marked out of scope, and an empty search never counts as evidence of no consequence.

8 The contribution cycle

Someone outside the original research can contribute a correction or take an argument further, and that work returns to the same record. The public repository publishes this research, checking and revision cycle as thirteen agent skills that install into common agent harnesses. Each skill owns one job, with its starting state, the changes it may make, the evidence it must return, its stopping condition and the next owner. The ordinary path orients (explain-public-system), works on one open obligation (mine-open-problem), validates (land-lean-proofs), propagates (propagate-research-consequences), packages the return with its prose held to the evidence (public-mathematical-writing) and proposes a pull request from a named starting commit (submit-pull-request). The [companion contribution protocol](#) gives the full sequence, and the [companion runtime paper](#) describes navigation and proof receipts in a fresh clone. Adoption replays the contributor's original change from its named starting commit before reconciling it with the current tree, so the original work and any conflict resolution are credited separately. A cycle closes when the change has evidence and every consequence has a recorded disposition; a problem closes only when a proof or a refutation settles its original statement.

Credit. The [credit policy](#) lets an idea be credited separately from its proof, its Lean formalisation and its explanation, the roles Kra asks journals to distinguish [5], and credits software as a role of its own. A first return can be a plain-language research-progress issue or email: no clone, proof, code or receipt schema is required. Accepted work receives a public receipt tied to exact artifacts. Acceptance does not establish theorem status, novelty, or release inclusion. Corrections append to the history, and an earlier contributor's record stays in it. Work also leaves the repository: Google DeepMind's Formal Conjectures maintainers have merged four of its contributions, linking formal proofs for Problems 257, 1049 and 1041 and a constant-base variant of Problem 258.

The public [contribution record](#) contains one accepted architecture return from 23 September: OpenAI Codex repaired the ordinary-language route into the research-return workflow and supplied a checked-positive architecture packaging case. The focused checks were reproduced by the same contributor in a maintainer-operated review. This is evidence that the credited local adoption protocol has been used. It records no independent outside clone replay and no mathematical endorsement.

8.1 Worked example: Problem 1041

Problem 1041 shows the cycle end to end, with the decisive idea arriving from outside. It asks whether, for a monic polynomial f with all roots in the open unit disc, two roots can always be joined inside the lemniscate $\{|f| < 1\}$ by a path of length less than 2 [20, 21]. On 7 September 2026 the erdosproblems.com contributor ani posted a degree-seven counterexample, found, as the post says, with the help of GPT-6 [22], and the record took the construction as an attributed input five days later. Lean first checked the total-variation form. On 22 September it proved

that for one polynomial of that construction every preconnected subset of the strict lemniscate containing two distinct roots has one-dimensional Hausdorff measure greater than 2 ([HausdorffLength.lean](#)), which refutes the exact path-image-length statement in Google DeepMind’s Formal Conjectures repository [23]. On 23 September the Formal Conjectures maintainers merged a change that marks the problem solved with answer `False` and links this proof [24]. The short paper and long record carry the proof with its Lean mark and credit `ani` for the construction. One judgement remains open: whether the formal statement matches the 1958 wording [21, Problem 5, p. 139]. The record says so beside the result.

9 Eight problems in one repository

The [public repository](#) maintains eight Erdős problems [1] as problem-sized worlds. Table 3 gives one result for each problem and the point where it stops.

The Lean library holds 1,903 modules, its two root files included, built under a pinned toolchain; the repository also holds sixteen problem papers, a cross-problem synthesis paper and three systems papers. The claim registry holds 160 public claims in eight statuses, from proved here and formalised here to conditional reduction and open, and states 19 remaining obligations exactly. Continuous integration builds the supported Lean roots, a coverage build compiles every module a paper cites, and the release checker checks claims, papers, generated views and licences. A fresh clone needs no private file, and no public theorem depends on an unpublished lemma.

The clone also carries the machinery for continuing a proof. A [proof-state compiler](#) takes the declarations that static queries nominate for a goal and asks the pinned Lean environment what each application does. A transition exists only when Lean reaches the state after the application; the subgoals Lean leaves form an AND set that becomes the exact remaining obligation, competing applications are OR alternatives, and a proof receipt exists only when Lean exits with no goals left. In the transfer of Section 4 both applications end with no remaining goals, the rejected one because Lean reached no state after it, so acceptance and closure are recorded separately.

The records come from a private environment, built over the past year, that runs agents from several model providers, including Anthropic’s Claude and OpenAI’s GPT models, concurrently on one shared file system. Durable state lives in files, and before an agent opens a large source a router compiles a packet conditioned on its task, with each item labelled by evidence class and a list of what was omitted. Agents claim exact paths for a bounded lease before changing them. Lean builds pass through a semantic single-flight queue keyed by source, targets, toolchain and dependency lock, so equivalent requests share one build, and a deferral is recorded separately from theorem failures. A failure in navigation, scheduling, experimentation or validation can change a route, skill or check for later agents through a guarded proposal, and it never changes the status of a mathematical claim. The public repository replays without the private environment: the private environment explains how the records were produced, and the public evidence is what supports each claim.

10 Relation to Prove2Me and other systems

Prove2Me [31] is the closest published system: a hosted platform on which anyone with a coding agent states Lean theorems and submits proofs. Each statement is immutable and may carry several proofs; the server accepts a proof only when its type is exactly the target’s, with no sorry and only whitelisted axioms, and a disproof proves the negation. Missions on papers, textbooks and open problems confine human audit to a goal, its definitions and milestone lemmas, aided by a blind agent read-back of each Lean statement, and a proof sketch may import open statements

Problem	A result to start with	Where it stops
#68, factorial denominator	Lean checks that the logarithm of the uncleared common denominator grows at least like $N^{3/2} \log N$, and that irrationality is equivalent to cofinally many non-unit factorial carries	Those carries are not produced
#243, Sylvester-tail rigidity	For a strictly increasing sequence of positive integers, Lean checks irrationality of $\sum 1/a_n$ under the exact cubic rate $a_n^2/a_{n+1} = 1 + 3/n + o(n^{-3})$	The unrestricted Sylvester-tail question
#249, binary totient series	For every base $k \geq 2$ and level $e \geq 1$, the totient sections $\varphi(k^j n + r)$ with $0 \leq j \leq e$ and $0 \leq r < k^j$ span a space of dimension $k^e + 1$ over $\mathbb{Q}$; certificates for every $t \leq 82$	Certificates for arbitrarily large t
#251, prime-gap dyadic series	Lean and Comparator check that sparse corrections preserving the congruences fill an interval of sums; from the prime number theorem and a lemma of Schlage-Puchta as named inputs, Lean checks a sequence with the local statistics of prime gaps and a rational dyadic sum	An argument specific to actual prime gaps
#257, infinite exponent supports	The weighted-support criterion of Section 1, at every integer base	Arbitrary infinite supports
#269, three-prime running LCMs	Lean and Comparator check that the three-prime running-LCM kernel has nonsingular minors of every order, so no finite sum of separated products represents it; both two-prime sums are transcendental by a cited Hecke–Mahler theorem, the repeated-sum case first posted by Fan [29]	The three-prime case
#1041, lemniscate connections	The Formal Conjectures statement is refuted with ani’s example	Match with the 1958 wording
#1049, rational-base Lambert series	Lean checks irrationality in Zudilin’s rational-base region and exact Hankel orders	Base 3/2 and all rational bases

Table 3: One result per problem, with its stopping point. The papers state each result in full with its hypotheses and sources.

for other agents to close. Its published examples also include proof explanations, discussion and a formalisation corrected after a disproof; its service documents statement deprecation and relinking. Persistence, correction, shared work and exact statement checking are therefore overlapping capabilities. The comparison is about what a reader can do with each public research object (Table 4). Prove2Me is the published reference for hosted statement and proof separation, audited mission cores and read-back auditing.

The two can compose: a Prove2Me sketch with imported open statements is a conditional argument for this record’s graph. Whether the surrounding record improves a reader’s decisions remains unmeasured. The Problem 1041 sharp constant, also compared by Comparator, is hosted on Prove2Me as 21 accepted theorems ported from Lean 4.29.1 to 4.30.0.

	Prove2Me	Problem-sized worlds
Unit of work	A mission on a paper, textbook or open problem, built from immutable statements and proofs	One research problem with its whole record
What persists	Statements, proofs, sketches, explanations, milestones and discussion; disproofs and deprecations stay visible	Short and long papers, failed routes, experiments, typed claims and open obligations beside the Lean source
Exact-statement check	Every submission, on the server	Selected statements, through Comparator, with a second kernel in the corpus replay
Statement fidelity	A captain audits each mission core with a blind read-back; a moderator approves public missions	The maintainer is responsible for the wording of each public claim in the claim registry; no independent review is recorded yet
Exposition	A description for each statement and an explanation for each proof	A short paper and a long record for each problem, with evidence marks at each result
Credit	Permanent names, trust scores, leaderboards and import citations	Receipts by contribution role, and attribution for outside ideas
Contribution path	Web platform and API, with an account	A plain-language issue or email without a clone; a clone and agent skills for source changes

Table 4: Prove2Me and problem-sized worlds, from Prove2Me’s paper and public pages and from this repository, on 24 September 2026.

Other systems. Feng et al. ran an agent over the 700 Erdős problems then listed as open, published graded outcomes [17], and propose reporting AI-assisted results on at least two axes, autonomy and significance, with significance judged only by mathematicians expert in the area [32]; the claim registry leaves significance to experts in the same way. Tao’s community wiki listed AI contributions to the Erdős problems, failed attempts included, until its updates stopped on 30 June 2026 [33]. Formal Conjectures [23] turns open problems into formal targets, and Ringer distinguishes mathematics as a process from mathematics as a benchmark [34]; a world keeps the process around such a target. Blueprints connect informal proof plans to Lean declarations [35]; LeanArchitect extracts blueprint data from annotated Lean source, infers dependencies and tracks progress [36]; and LeanMarathon maintains an evolving blueprint, one Lean file serving as proof skeleton, natural-language proof graph and shared record, worked by separate construction, audit, proof and repair agents [37]. These precede durable formal development linked to explanation. Hattori, Matsuzaki and Fujiwara translate and summarise Lean proofs along their structure [38], a direct precedent for the explanation a world keeps beside a formal proof. This record adds appraisal and failed routes around each proof. A blueprint maps planned or source-inferred dependencies; the argument graph also searches unplanned connections across a record. Provers such as ReProver, built on LeanDojo, and OpenProver [40, 41] can supply a world with candidate proofs, and Li et al. hold generation and validation apart in research mathematics without a proof assistant [42].

Precedents for the argument graph. Gandhi, Tadipatri and Gowers generalise a Lean proof about a particular constant: given a proof that $\sqrt{17}$ is irrational, their tactic finds that the proof uses only that 17 is prime and returns the theorem that $\sqrt{n}$ is irrational for every prime n , with its proof [44]. The used-consequence pass asks what a proof uses of a hypothesis and searches those statements across the corpus. The unusedArguments linter of Batteries, which Mathlib runs over its library, reports the arguments a declaration never uses [45]; the idle pass applies that test to the proposition binders of every theorem in the record, has the kernel accept each theorem without them, and adds the stronger statement to the graph. AXLE, a service of Lean metaprogramming tools, removes tactics and have steps a proof does not use, lifts the obligations inside a partial proof into standalone lemmas, and checks a proof against a separately given statement and a list of permitted axioms [46]. The used-consequence pass lifts statements out of a proof in a similar way, and Comparator makes a check of the last kind for this record. Proof mining extracts from a given proof information that its statement does not record, such as explicit bounds [43]; the factoring pass reads off a proof which claims of its conclusion each hypothesis supports, and the kernel checks the statement that records it.

Li, Peng, Severini and Shafto analyse Mathlib as a multilayer network of declarations and modules, measuring centrality, communities and the effect of removing a node [47], and Huch reports preliminary evidence of a scale-free in-degree distribution in the dependency graph of the Isabelle Archive of Formal Proofs [48]. The argument graph's nodes are the closed statements of a research record and its edges the reductions the unifier finds; leverage and criticality ask what supplying or withdrawing one statement would change. De Kleer's assumption-based truth maintenance system records justifications as propositional Horn clauses and labels each node with the minimal consistent sets of assumptions from which it follows [49]. The graph's reductions are Horn clauses over closed statements; bundles are minimal sets of open premises, enumerated up to four members. A joint query checks for conflict (a nogood) or a restatement of a target over recorded edges; the graph keeps no complete set of minimal inconsistent sets. Mokhov, Mitchell and Peyton Jones describe a build system by the order in which it runs tasks and how it decides, from records of earlier builds, whether a task must run again [50]; the graph is rebuilt from a complete export and records the Lean tree it read, so the summary query reports it stale once that tree changes.

Zhu, Clune, Avigad, Jiang and Welleck build LeanHammer from a trained premise selector, translation into external automatic provers and proof reconstruction in Lean [51]. The producer search chooses its candidates by the head symbol and the constants of their conclusions, and follows what each candidate leaves unproved. Aesop runs a best-first proof search over a configurable set of rules [52], and Plausible tests a proposition on generated examples and reports a counterexample [53]. The battery runs a fixed list of closing tactics and library search, and the graph refutes a statement only through kernel-checked theorems.

Libraries grown from proofs. Kaliszyk and Urban rank intermediate statements of HOL Light proofs by reuse and test the ranking with a prover, controlling what each theorem's own proof leaks [55]. Hipster keeps a lemma only when a routine tactic fails on it [56]; DreamCoder keeps a routine only when rewriting solved programs through it shrinks the whole [57]. LEGO-Prover retrieves verified lemmas by requested subgoals [58]; at matched compute its proofs reuse none verbatim [63]. Proof-Refactor lifts proof blocks into helpers that a language model judges [64], LeanConjecturer screens conjectures with exact? and Aesop [65], and Georgiev et al. report scores met without the property measured [66]. DreamProver grows a Lean lemma library in wake and sleep stages and ablates the whole library [59], as CircuitProver does for hardware proofs [60]; ProofEvolve stores kernel-checked sub-proofs as schemas whose unmet premises become subgoals [61]; Böhme and Nipkow minimise the facts of one proof by withdrawing each and rechecking [62]. Here the gain of a restatement is a kernel-checked theorem, and its use by another proof is read from proof terms and from what withdrawing it loses.

11 Limits, and what stronger models change

The other seven targets remain open, and independent human review of the correspondence between the Problem 1041 refutation and the 1958 wording has not been recorded. The system records evidence classes and review events; novelty and significance are judgements it leaves to experts, as Feng et al. also propose [32]. The examples here show how the record organises results; whether the workflow made those results more likely has not been measured. By 28 September 2026 no outside human contributor had opened a pull request or issue in the repository; its outside record is the Formal Conjectures merges of Section 8 and the Prove2Me package. A record also leaves several of the concerns in Section 1 where it finds them: it trains no one, it restores no signal of individual expertise [67], and it answers neither the ethical objections some mathematicians raise to AI-assisted mathematics [68] nor the warning of forty-two Fellows and Foreign Members of the Royal Society about catastrophic risk from AI [69].

The same design serves stronger models. A stronger model starts from the current frontier with the failed routes already marked, and it can reorganise, re-explain and extend the record as well as the mathematics. The public interfaces admit any producer: an outside contributor can attach any agent runner to the clone, and a laboratory with many frontier models could replace the private production environment while keeping the same evidence boundary.

Gowers suggests a well-designed database of what is known, probably built with AI help, and adds that it may prove unnecessary if one can ask a language model for a bird's-eye view of an area [70]; Koukoulopoulos proposes a public facility in which coordinated agents explore a research programme divided into subproblems under researchers' direction [71]. A problem-sized world is one concrete way to maintain the sources, explanations and open questions such work draws on, here at the scale of eight problems, and its argument graph is one concrete form for the map such work needs: its subproblems are the open statements, and the edges between them are corpus theorems matched by unification. Antieau writes that texts generated largely by a language model deserve a home other than the arXiv [72]; a world gives its long records one beside the proofs they explain.

The graph gives the record a measurement of its own. Each build is dated by the revision it was computed from, and the difference between two builds lists which open statements a change settled, which disguise classes it merged and which conditional theorems it turned unconditional, so what a landed theorem did is read from the proof terms. Whether a record organised this way makes understanding, checking and continuing a result less work for other people is a question for its users; no comparison with other organisations of the same material has been run.

12 Conclusion

A problem-sized world keeps the record that lets other people understand a proof, check it, credit it and build on it: its mechanism, its antecedents, its formal evidence and its exact open statements. The weighted criterion, the totient basis and the derivative-interpolation theorem show that record for three results, the dyadic transfer shows the corpus supplying a proof its parts did not contain, and the eight worlds in the public repository show that it can be kept persistent, checkable and open to contribution across several hard problems. The argument graph turns the record's unfinished arguments into a map of its problems: what is proved, what is refuted, what reduces to what, what is the open problem in other coordinates and what each missing input would settle, with every kernel-checked row a proof and the rest a lower bound that says so. The research record keeps custody of the answers returned in the rounds it records and computes what each still owes. The next empirical question is whether source-bound packets improve exact reader decisions under controlled conditions, followed separately by whether

reviewed mechanisms help an independent consumer. Neither effect follows from the size of this record.

A Inspection routes and reproducibility

Question	Start here
How does the repository fit together?	docs/ARCHITECTURE.md
What is proved and what is open?	docs/RESULTS.md and docs/claims.json
Which declaration states each paper result?	docs/paper_lean_coverage.json
Where is the checked mathematics?	lean/Erdos249257.lean and lean/ErdosProblems.lean
What does Comparator check?	docs/EXTERNAL_VERIFICATION.md , verification/comparator.json and the corpus replay index evidence/comparator/associations.json
Whose earlier work does each result use?	docs/PRIOR_ART.md
Which checks gate a release?	scripts/check_release.py and .github/workflows/lean.yml
How can work return with credit?	CONTRIBUTING.md , credit policy and architecture contributions
Which agent skills exist?	skills/README.md

Table 5: Entry points for inspection. They lead to the authority-bearing files and add no authority of their own.

A fresh clone can inspect the control card and check its committed navigation without Lean:

```
python3 scripts/proof_cockpit.py --format card
python3 scripts/proof_cockpit.py --check
```

Release-surface validation is a separate operation using the pinned Python requirements in [the reproduction guide](#); it runs `python3 scripts/check_release.py`. Formal replay additionally uses the pinned Lean and Comparator environment named by the card and guide. The paper inventory, [docs/publication_contract.json](#), records source and PDF hashes and validation commands, and the evidence for the checker study in Section 7 is recorded in [docs/publication_evidence.json](#).

References

- [1] T. F. Bloom, *Erdős problems*, [erdosproblems.com](#), accessed September 2026.
- [2] W. Cook, *Weighted Support Criteria for Reciprocal Mersenne Subseries*, short paper on Erdős Problem 257, September 2026, [PDF](#).
- [3] P. Erdős, *On the irrationality of certain series*, Math. Student 36 (1968), 222–226 (issued 1969), [scan](#).
- [4] D. Duverney and Y. Tachiya, *Refinement of the Chowla–Erdős method and linear independence of certain Lambert series*, Forum Math. 31 (2019), 1557–1566, [DOI](#).
- [5] B. Kra, *Deep theorems were scarce and difficult and so became an effective mechanism to identify deep thought. AI has broken this system*, guest post on *What’s new*, 13 September 2026, [blog post](#).
- [6] H. Cohn, *The technical debt of AI-generated mathematics*, guest post on *What’s new*, 15 September 2026, [blog post](#).
- [7] T. Tao, thread on mining open problems, Mathstodon, 8 September 2026, [thread](#).
- [8] Twenty-five Fields Medallists, *A severe misalignment of AI in mathematics*, declaration posted on *What’s new*, 11 September 2026, [blog post](#).

- [9] G. Sanderson, *If math is more than proof, we need to better celebrate the rest of it*, guest post on *What's new*, 18 September 2026, [blog post](#).
- [10] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, [DOI](#).
- [11] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, 1980, p. 61.
- [12] W. Cook, *Bases and Integral Relations for the k -Kernel of Euler's Totient*, short paper on Erdős Problem 249, revised 18 September 2026, [PDF](#).
- [13] M. Coons, *(Non)Automaticity of number theoretic functions*, J. Théor. Nombres Bordeaux 22 (2010), 339–352, [DOI](#), Theorem 3.2.
- [14] G. Martin, *Simultaneous inequalities among values of the Euler phi-function*, 2006, [arXiv:math/0603053v1](#), Theorem 1.
- [15] S. Yazdani, *Multiplicative functions and k -automatic sequences*, J. Théor. Nombres Bordeaux 13 (2001), 651–658, [Numdam](#).
- [16] W. Cook, *Reading Eight Erdős Problems Together*, synthesis paper, September 2026, [source](#), including the [periodic-chain theorem](#).
- [17] T. Feng, T. Trinh, G. Bingham, et al., *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*, 2026, [arXiv](#).
- [18] Lean FRO, *Comparator*, 2025, [GitHub](#).
- [19] ammkrn, *nanoda_lib*, an independent type checker for Lean 4, [GitHub](#).
- [20] T. F. Bloom, *Erdős Problem #1041*, [erdosproblems.com](#), accessed September 2026.
- [21] P. Erdős, F. Herzog and G. Piranian, *Metric properties of polynomials*, J. Analyse Math. 6 (1958), 125–148, [DOI](#).
- [22] ani, post in the Problem 1041 discussion thread, 7 September 2026, [erdosproblems.com forum](#).
- [23] Google DeepMind, *Formal Conjectures*, [GitHub](#), accessed September 2026.
- [24] *Formal Conjectures*, pull request 6505, *Erdős 1041: mark solved with answer(False) and link a formal proof*, merged 23 September 2026, [GitHub](#).
- [25] T. Zheng, X. Wu, Z. Zhang, et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, 2026, [arXiv:2609.14858](#).
- [26] J. H. Loxton and A. J. van der Poorten, *Arithmetic properties of certain functions in several variables III*, Bull. Austral. Math. Soc. 16 (1977), 15–47.
- [27] Y. Bugeaud and M. Laurent, *Transcendence and continued fraction expansion of values of Hecke–Mahler series*, Acta Arith. 209 (2023), 59–90.
- [28] W. Cook, *The Three-Prime Running LCM*, short paper on Erdős Problem 269, September 2026, [PDF](#).
- [29] S. Fan, comment on Erdős Problem #269, [erdosproblems.com forum](#), 26 June 2026, [forum post](#).
- [30] N. Patel, A. Rammal, A. Hayat, R. Munos, and J. Kempe, *Learning to Discover Interesting Mathematics*, [arXiv:2609.28603v1](#), 2026, <https://arxiv.org/abs/2609.28603>.
- [31] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng, *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*, 2026, [arXiv](#).
- [32] T. Feng, T. H. Trinh, G. Bingham, et al., *Towards Autonomous Mathematics Research*, 2026, [arXiv](#).
- [33] T. Tao, *AI contributions to Erdős problems*, [GitHub](#), accessed September 2026.
- [34] T. Ringer, *Becoming a benchmark*, guest post on *What's new*, 17 September 2026, [blog post](#).
- [35] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, [software repository](#).
- [36] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16, [DOI](#).
- [37] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, [arXiv:2606.05400](#).
- [38] S. Hattori, T. Matsuzaki, and M. Fujiwara, *Natural Language Translation of Formal Proofs through Informalization of Proof Steps and Recursive Summarization along Proof Structure*, in *Proceedings of the 18th International Natural Language Generation Conference*, 2025, pp. 376–389, [ACL Anthology](#).
- [39] S. Akiyama and H. Kaneko, *Multiplicative analogue of Markoff–Lagrange spectrum and Pisot numbers*, Adv. Math. 380 (2021), 107547, p. 3, reporting the theorem of A. Dubickas, J. Number Theory 117 (2006), 222–239, [arXiv:1911.06170v6](#).
- [40] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023, Datasets and Benchmarks Track, [arXiv:2306.15626](#).
- [41] M. Kripner and M. Straka, *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*, 2026, [arXiv](#).
- [42] C. Li, Z. Lai, D. An, J. Hu, and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, 2025, [arXiv:2512.09443v2](#).
- [43] U. Kohlenbach, *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*, Springer Monographs in Mathematics, Springer, Berlin, 2008.
- [44] A. Gandhi, A. R. Tadipatri, and T. Gowers, *Automatically Generalizing Proofs and Statements*, in *16th International Conference on Interactive Theorem Proving*, LIPIcs 352, 2025, pp. 12:1–12:18, [DOI](#).
- [45] Lean community, *Batteries*, the unusedArguments linter in Batteries/Tactic/Lint/Misc.lean, [GitHub](#), accessed September 2026.
- [46] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh, and J. Limperg, *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*, 2026, [arXiv:2606.26442](#).
- [47] X. Li, N. Peng, S. Severini, and P. Shafto, *The Network Structure of Mathlib*, 2026, [arXiv:2604.24797](#).

- [48] F. Huch, *Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs*, extended abstract, 2022, [arXiv:2209.13305](#).
- [49] J. de Kleer, *An assumption-based TMS*, Artificial Intelligence 28 (1986), 127–162, [DOI](#).
- [50] A. Mokhov, N. Mitchell, and S. Peyton Jones, *Build Systems à la Carte*, Proc. ACM Program. Lang. 2 (ICFP), 2018, article 79, [DOI](#).
- [51] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck, *Premise Selection for a Lean Hammer*, 2025, revised 2026, [arXiv:2506.07477](#).
- [52] J. Limperg and A. H. From, *Aesop: White-Box Best-First Proof Search for Lean*, in *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2023, pp. 253–266, [DOI](#).
- [53] Lean community, *Plausible*, a property testing framework for Lean 4, [GitHub](#), accessed September 2026.
- [54] H. Chockler, A. Gurfinkel, and O. Strichman, *Beyond Vacuity: Towards the Strongest Passing Formula*, Form. Methods Syst. Des. 43 (2013), 552–571, [DOI](#).
- [55] C. Kaliszyk and J. Urban, *Lemma Mining over HOL Light*, LPAR-19, LNCS 8312, 2013, pp. 503–517, [DOI](#).
- [56] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen, *Hipster: Integrating Theory Exploration in a Proof Assistant*, CICM 2014, LNCS 8543, pp. 108–122, [DOI](#).
- [57] K. Ellis et al., *DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning*, 2020, [arXiv:2006.08381](#).
- [58] H. Wang, H. Xin, et al., *LEGO-Prover: Neural Theorem Proving with Growing Libraries*, ICLR 2024, [arXiv:2310.00656](#).
- [59] Y. Zhang, J. Sun, et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent*, 2026, [arXiv:2604.26311](#).
- [60] Z. Yang, W. Fang, et al., *CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library for Hardware Verification*, 2026, [arXiv:2607.27259](#).
- [61] W. Ye, Z. Guan, et al., *ProofEvolve: Neuro-Symbolic Evolution for Formal Automated Theorem Proving*, 2026, [arXiv:2608.26334](#).
- [62] S. Böhme and T. Nipkow, *Sledgehammer: Judgement Day*, IJCAR 2010, LNCS 6173, pp. 107–121, [DOI](#).
- [63] I. Berlot-Attwell, T. Sesterhenn, F. Rudzicz, and X. Si, *Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour*, EACL 2026, pp. 3534–3568, [DOI](#).
- [64] Y. Fu, P. Liu, Z. Wang, and K. Yuan, *Proof-Refactor: Refactoring Generated Formal Proofs into Modular Artifacts*, 2026, [arXiv:2606.03743](#).
- [65] N. Onda et al., *LeanConjecturer: Automatic Generation of Mathematical Conjectures for Theorem Proving*, 2025, [arXiv:2506.22005](#).
- [66] B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner, *Mathematical Exploration and Discovery at Scale*, 2025, [arXiv:2511.02864v3](#).
- [67] D. Litt, *A beginning for mathematics*, 13 September 2026, [blog post](#).
- [68] T. Chu, *The AI dissenter viewpoint, Proofs and Prompts*, 9 August 2026, [blog post](#).
- [69] B. Green and forty-one other Fellows and Foreign Members of the Royal Society, *Open letter to Sir Paul Nurse, President of the Royal Society*, Proofs and Prompts, 17 September 2026, [blog post](#).
- [70] W. T. Gowers, *Why I didn't sign the Fields medallists' letter*, 17 September 2026, [blog post](#).
- [71] D. Koukoulopoulos, *A CERN for AI-assisted science?*, guest post on *What's new*, 17 September 2026, [blog post](#).
- [72] B. Antieau, *Fast math/slow math*, 15 September 2026, [blog post](#).